AULA 06
ESTRUTURAS DE DADOS

Programação em Python • Parte 2 de 2
Curso de Capacitação em Desenvolvimento Full Stack
Tema: Estruturas de Dados • 8 horas
Professor: Msc. Hygo Sousa de Oliveira

(95) 99173-5676 / (92) 99171-0391
www.iteam.org.br
Av. Glaycon de Paiva nº 1820,
Mecejana
CEP: 69.304-560
Boa Vista/RR
CNPJ: 48.653.884/0001-00

Roteiro da aula

O que vamos ver nesta aula

1. Compreensões de lista, conjunto e dicionário
2. Funções lambda
3. map(), filter() e reduce()
4. Exemplos guiados
5. Do if/elif ao padrão Registry
6. Checkpoint e atividades

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Criar listas, conjuntos e dicionários com compreensões**
  Uma linha, clara e pythônica
- **Escrever funções lambda**
  Funções anônimas para tarefas curtas
- **Usar map(), filter() e reduce()**
  Transformar, selecionar e combinar elementos
- **Substituir if/elif por um dicionário de funções**
  Primeiro contato com o padrão Registry

List comprehension

Criar listas em uma única expressão

list_comp.py

```python
# forma tradicional
dobrados = []
for n in range(1, 6):
    dobrados.append(n * 2)

# list comprehension
dobrados = [n * 2 for n in range(1, 6)]
pares = [n for n in range(10) if n % 2 == 0]
print(dobrados, pares)
```

SAÍDA

```
[2, 4, 6, 8, 10] [0, 2, 4, 6, 8]
```

NOTA Sintaxe: `[expressão for item in iterável if condição]`. O `if` é opcional.

Set e dict comprehension

A mesma sintaxe, com chaves

set_dict_comp.py

```
numeros = [1, 2, 3, 4, 5, 6]
paridades = {n % 2 for n in numeros}
quadrados = {n: n ** 2 for n in range(1, 4)}
invertido = {v: k for k, v in quadrados.items()}
print(paridades)
print(quadrados)
print(invertido)
```

SAÍDA

```
{0, 1}
{1: 1, 2: 4, 3: 9}
{1: 1, 4: 2, 9: 3}
```

DICA No conjunto, as duplicatas somem sozinhas. No dicionário, informe `chave: valor`.

Funções lambda

Funções anônimas, compactas e versáteis

lambda.py

```
soma = lambda x, y: x + y
quadrado = lambda x: x ** 2
print(soma(5, 3), quadrado(3))
pessoas = [("Ana", 28), ("Bruno", 21), ("Carla", 35)]
print(sorted(pessoas, key=lambda p: p[1]))
```

SAÍDA

```
8 9
[('Bruno', 21), ('Ana', 28), ('Carla', 35)]
```

NOTA Sintaxe: `lambda argumentos: expressão`. Uma lambda tem uma única expressão e devolve o resultado.

map(), filter() e reduce()

Os pilares da programação funcional no Python

Função	O que faz
`map()`	Aplica uma função a cada elemento
`filter()`	Seleciona elementos que satisfazem uma condição
`reduce()`	Combina todos os elementos em um único valor

funcional.py

```
from functools import reduce
numeros = [1, 2, 3, 4]
dobros = list(map(lambda x: x * 2, numeros))
pares = list(filter(lambda x: x % 2 == 0, numeros))
total = reduce(lambda x, y: x + y, numeros)
print(dobros, pares, total)
```

SAÍDA

```
[2, 4, 6, 8] [2, 4] 10
```

Comprehension ou map/filter?

Mesmo resultado, legibilidade diferente

comparacao.py

```
numeros = [1, 2, 3, 4]
a = [x * 2 for x in numeros if x % 2 == 0]
b = list(map(lambda x: x * 2, filter(lambda x: x % 2 == 0, numeros)))
print(a == b, a)
```

SAÍDA

```
True [4, 8]
```

DICA Na maioria dos casos a comprehension é mais direta e mais fácil de ler. Reserve `map`/`filter` para quando já houver uma função pronta.

Exemplo guiado 1: preços com desconto

Dicionário e lista a partir de um dicionário

DESAFIO Aplique 10% de desconto em todos os produtos e liste os que custam mais de R$ 10.

precos.py

```
produtos = {"caneta": 2.5, "caderno": 18.0, "mochila": 120.0}
com_desc = {n: p * 0.9 for n, p in produtos.items()}
caros = [n for n, p in produtos.items() if p > 10]
print(com_desc)
print(caros)
```

SAÍDA

```
{'caneta': 2.25, 'caderno': 16.2, 'mochila': 108.0}
['caderno', 'mochila']
```

Exemplo guiado 2: ranking de alunos

sorted() com lambda e filtro por comprehension

DESAFIO Ordene os alunos da maior para a menor nota e liste quem tem nota 7 ou mais.

ranking.py

```
alunos = [("Ana", 9.0), ("Bruno", 6.5), ("Carla", 8.0)]
ranking = sorted(alunos, key=lambda a: a[1], reverse=True)
for pos, (nome, nota) in enumerate(ranking, start=1):
    print(f"{pos}º {nome} - {nota}")
print([n for n, nota in alunos if nota >= 7])
```

SAÍDA

```
1º Ana - 9.0
2º Carla - 8.0
3º Bruno - 6.5
['Ana', 'Carla']
```

O problema do if/elif crescente

Cada nova operação exige mexer no código existente

antes.py

```
operacao, a, b = "soma", 8, 2
if operacao == "soma":
    resultado = a + b
elif operacao == "subtracao":
    resultado = a - b
elif operacao == "multiplicacao":
    resultado = a * b
else:
    resultado = None
print(resultado)
```

SAÍDA

```
10
```

ATENÇÃO Com 5, 10, 20 operações a cadeia só cresce, fica acoplada e difícil de testar e de estender.

Registry: um dicionário de funções

Registrar comportamentos em vez de encadear condições

depois.py

```
operacoes = {
    "soma": lambda a, b: a + b,
    "subtracao": lambda a, b: a - b,
    "multiplicacao": lambda a, b: a * b,
}
operacao, a, b = "soma", 8, 2
funcao = operacoes.get(operacao)
print(funcao(a, b) if funcao else "Operação inválida")
operacoes["divisao"] = lambda a, b: a / b
print(operacoes["divisao"](8, 2))
```

SAÍDA

```
10
4.0
```

DICA Nova operação = nova entrada no dicionário, sem tocar na lógica que escolhe e executa.

Atividades da Aula 6

Módulo 3: checkpoint e início do Projeto Registry (N1)

- **Projeto Registry Pattern (N1) – Atividade 1**
  Calculadora de operações com dicionário e lambdas (nível básico)
- **Checkpoint de Estruturas de Dados (10 questões)**
  Listas, tuplas, dicionários, conjuntos e comprehensions; entrega via GitHub

GIT Enunciados, instruções de execução e de entrega:
github.com/Hygo/python-fullstack-iteam-2sem

Obrigado(a)!

Próxima aula: Funções e Módulos (definição, parâmetros e escopo)
ITEAM – Instituto Tecnológico Educacional da Amazônia

(95) 99173-5676 / (92) 99171-0391
www.iteam.org.br
Av. Glaycon de Paiva nº 1820, Mecejana
CEP: 69.304-560
Boa Vista/RR
CNPJ: 48.653.884/0001-00